

Below is a **print-ready document-style explanation** you can paste into Word, Markdown, Notion, etc. and export to PDF. I kept it visual, concise, and centered around the exact variables in your function.

Visual Walkthrough:

`image_to_parallel_lines`

What this function does

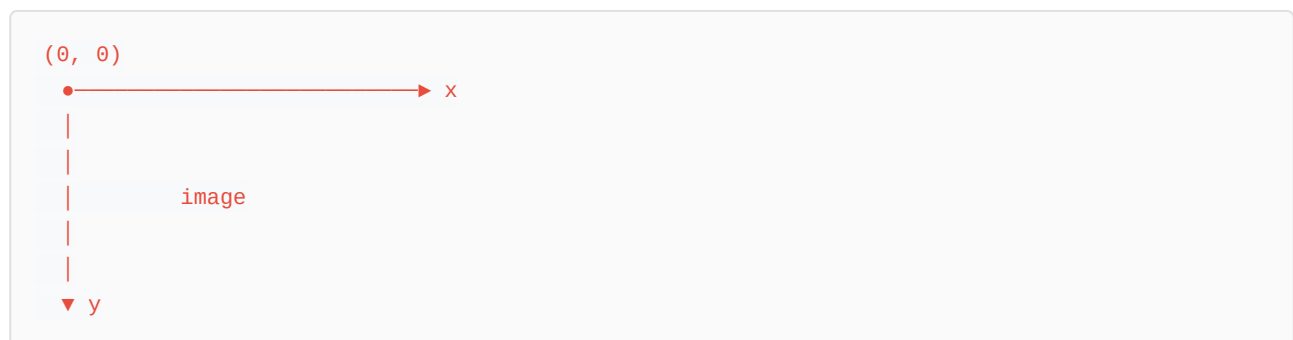
The function turns an image into **evenly spaced parallel line segments**.

It first creates many invisible guide lines across the image. Then it walks along each guide line and checks the original image underneath. Where the original image is dark enough, the line becomes visible. Where it is too light, the line is broken.

The result is a portrait made from many parallel black line segments.

1. The image coordinate system

Images use a coordinate system with the origin in the **top-left corner**.



So:

- `x` increases from left to right.
- `y` increases from top to bottom.
- The image has:
 - `width`

- `height`



2. The center point

The function uses the center of the image as the anchor point for building the parallel line system.

```
cx = width / 2
cy = height / 2
```

Visually:



This center point is not necessarily drawn. It is just the reference point from which the guide lines are placed.

3. The line direction: `dx`, `dy`

The parameter:

```
angle
```

controls the direction of all lines.

The code converts it into a direction vector:

```
dx = cos(angle)
dy = sin(angle)
```

Together, `(dx, dy)` describes the direction in which each line runs.

```
Direction vector:
(dx, dy)

line direction
  ↘
  ↘
  ↘
```

Important: because image `y` goes downward, positive angles rotate visually clockwise.

Examples:

```
angle = 0°
```

```
—————→
horizontal lines
```

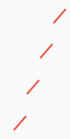
```
angle = 45°
```

```
 \
  \
   \
    \
down-right direction
```

```
angle = 90°
```

```
|
|
|
▼
vertical downward direction
```

```
angle = 135°
```



```
down-left direction
```

So:

```
(dx, dy) = direction along the line
```

4. The perpendicular direction: `nx`, `ny`

To create many parallel lines, the function needs a second direction: one that moves **sideways** from one line to the next.

That is the perpendicular vector:

```
nx = -sin(angle)  
ny = cos(angle)
```

If `(dx, dy)` points along the line, then `(nx, ny)` points across the lines.

Line direction:



Perpendicular direction:



Think of it like this:

```
(dx, dy) = walk along a line  
(nx, ny) = move to the neighbouring line
```

5. Parallel guide lines using `offset`

The function creates many invisible guide lines.

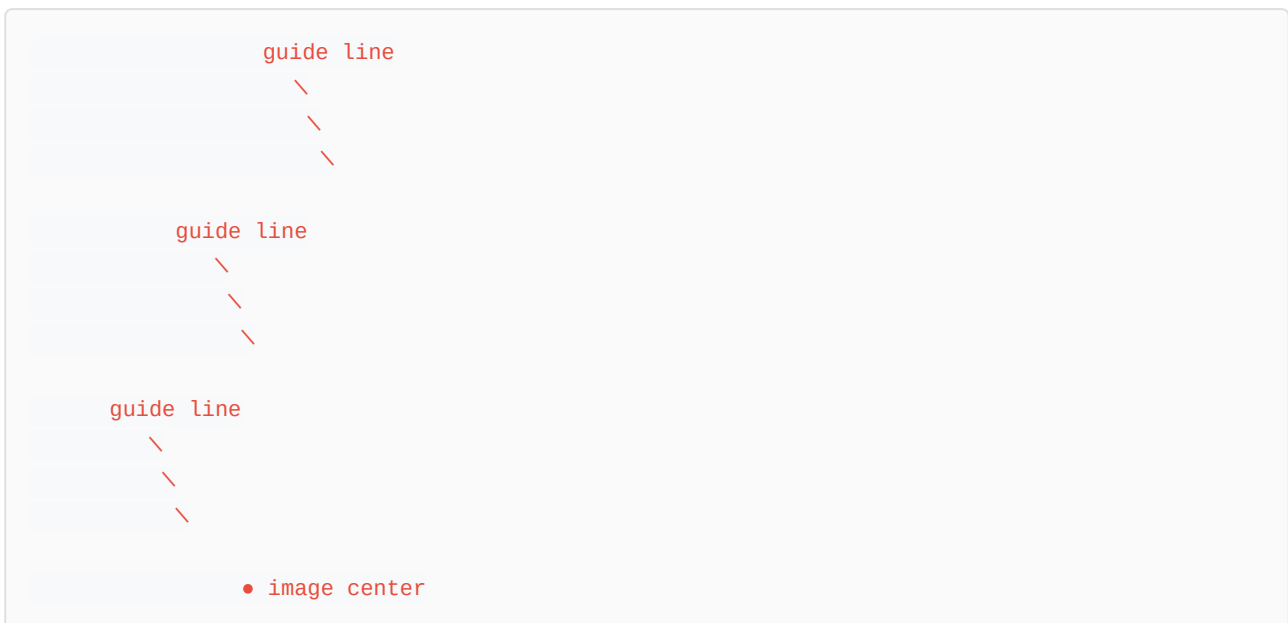
Each guide line starts from a base point:

```
base_x = cx + offset * nx
base_y = cy + offset * ny
```

So:

- start at the image center `(cx, cy)`
- move sideways using `(nx, ny)`
- the amount you move is `offset`

Visually:



The offsets create evenly spaced parallel lines:

```
offset = -24
offset = -16
offset = -8
offset = 0
offset = 8
offset = 16
offset = 24
```

passes near center

The distance between offsets is controlled by:

```
line_spacing
```

So:

```
line_spacing = distance between neighbouring guide lines
```

Smaller spacing:

Larger spacing:

6. Why the function uses `diagonal`

The function uses:

```
diagonal = sqrt(width2 + height2)
```

This gives a distance large enough to cover the full image, even when lines are rotated.

A rotated line may need to start outside the image and pass through it. That is why the function samples from:

```
t = -diagonal
```

```
t = +diagonal
```

This makes sure each guide line fully crosses the image.

7. Sampling along a line using `t`

For every guide line, the function walks along it.

Each sampled point is calculated as:

```
x = base_x + t * dx
y = base_y + t * dy
```

This means:

```
base point + distance along the line
```

Visually:

One guide line:

```
x      x      x      x      x      x
 \     \     \     \     \     \
  \     \     \     \     \     \
```

A simpler view:

sampled points along one line:

```
x—x—x—x—x—x—x—x—x
```

The distance between these sampled points is controlled by:

```
sample_step
```

Small `sample_step` :

```
x-x-x-x-x-x-x-x-x-x
more detailed
```

Large `sample_step` :

x — x — x — x
more coarse

8. Mapping sample points to image pixels

The calculated sample point (x, y) can be a decimal value.

The function rounds it to the nearest image pixel:

```
ix = round(x)
iy = round(y)
```

Visually:

```
continuous line point:
    • (x, y)

nearest image pixel:
    ■ (ix, iy)
```

Then it checks whether that pixel is inside the image:

```
inside image    → check darkness
outside image   → not visible
```

```
+-----+
|      x  x  x      |
|    x  x  x  x    |
|  x  x  x  x  x  |
+-----+
```

samples outside the box are ignored

9. Brightness and darkness

The original image is converted to grayscale.

Each pixel has a brightness value:

```
0    = black  
255  = white
```

The function converts brightness into darkness:

```
darkness = 1 - brightness / 255
```

So:

```
black pixel:  
brightness = 0  
darkness   = 1.0
```

```
white pixel:  
brightness = 255  
darkness   = 0.0
```

```
gray pixel:  
brightness = 128  
darkness   ≈ 0.5
```

10. Darkness threshold

The parameter:

```
darkness_threshold
```

decides whether a sampled point becomes visible.

```
visible = darkness >= darkness_threshold
```

Example:

```
darkness_threshold = 0.35
```

sample darkness values:

0.10	0.42	0.56	0.61	0.20	0.18	0.48	0.51
no	yes	yes	yes	no	no	yes	yes

Visually:

sample points:

x—x—x—x—x—x—x—x

visibility:

. ■ ■ ■ . . ■ ■

Where:

■ = visible
· = invisible

Lower threshold:

more samples become visible

Higher threshold:

only darker areas become visible

11. How line segments are formed

The function collects consecutive visible samples into a segment.

Example along one guide line:

visibility:

. . ■ ■ ■ . . ■ ■ ■ ■ . ■ .

This becomes:

```
segment 1:  █ █ █
segment 2:      █ █ █ █
segment 3:          █
```

Drawn as lines:

```
  _____
      _____
                        too short, maybe skipped
```

The function stores visible points in:

```
current_segment
```

When it hits an invisible sample, it ends the current segment.

12. Minimum segment length

Very tiny line fragments can look noisy.

That is why the function checks:

```
min_segment_length
```

If a segment is shorter than this value, it is not drawn.

```
Before filtering:
_  _ _ _ _  -  _ _ _ _ _  _
```

After filtering:

```
  _ _ _ _ _  _ _ _ _ _
```

So:

```
min_segment_length = removes tiny fragments
```

Small value:

```
many details, more noise
```

Large value:

```
cleaner image, fewer fragments
```

13. Final drawing process

For each visible segment, the function draws one line:

```
start point          end point  
  ●──────────────────●
```

The line thickness is controlled by:

```
line_width
```

```
line_width = 1
```

```
_____
```

```
line_width = 3
```

```
=====
```

All segments together form the final image:

```
Original dark regions:  
    face / shadows / hair
```

```
become:
```

```
parallel visible line segments
```

Example final structure:



The portrait appears because dark areas contain more surviving line segments.

14. Parameter cheat sheet

Parameter	Visual meaning	If you increase it
<code>angle</code>	Direction of all parallel lines	Lines rotate
<code>line_spacing</code>	Distance between neighbouring guide lines	Fewer, more separated lines
<code>sample_step</code>	Distance between sample points along a line	Rougher segment boundaries
<code>darkness_threshold</code>	Minimum darkness needed to show a line	Only darker regions remain
<code>min_segment_length</code>	Minimum length of a visible segment	Tiny fragments disappear
<code>line_width</code>	Thickness of drawn segments	Lines become heavier

15. Mental model of the whole function

The full process:

```

1. Start with image
  |
  ▼
2. Convert to grayscale
  |
  ▼
3. Create many parallel guide lines
  |
  ▼
4. Walk along every guide line
  |
  ▼
5. Check darkness at each sample point
  |
  ▼
6. Keep dark-enough samples
  |
  ▼
7. Group consecutive visible samples
  |
  ▼
8. Draw each group as a line segment

```

Visually:

Original image:

```

+-----+
|   face   |
| shadows  |
|   hair   |
+-----+

```

Invisible guide lines:

```

\ \ \ \ \ \ \ \

```

Darkness test:

only parts over dark pixels survive

Final result:

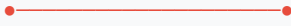
```

  \\\\
 \\\
  \\\
 \\\

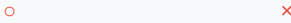
```

16. Future colouring-page idea

Right now, a segment is drawn like this:



Later, instead of drawing the full line, you could draw only markers:



Where:

○ = start point
× = end point

Then the person colouring or drawing has to connect the markers. If all the line segments are completed, the portrait gradually appears.

That future version would reuse the same segment logic. The only difference is the final drawing step:

current version:
draw full segment

future version:
draw only start and end markers

17. Quick visual summary

```
angle
controls this:
  \ \ \ \

line_spacing
controls this:
  \   \   \   \

sample_step
controls this:
  x—x—x—x—x
```

```
darkness_threshold
  controls this:
    keep dark samples, remove light samples

min_segment_length
  controls this:
    remove tiny broken bits

line_width
  controls this:
    thin line vs thick line
```

18. One-sentence summary

The function lays evenly spaced parallel guide lines over the image, samples brightness along those lines, keeps only the parts that pass a darkness threshold, groups consecutive visible samples into segments, and draws those segments as black lines.